

CreatePage.jsx

```
import { pageTemplates } from "../templates/pageTemplates";

import { useEffect, useState } from "react";

const CreatePage = () => {

  const [menus, setMenus] = useState([]);

  const [formData, setFormData] = useState({
    title: "",
    slug: "",
    parentSlug: "",
    pageType: "generic",
    status: "published",
    sections: structuredClone(pageTemplates.generic),
  });

  // FETCH NAVIGATION MENU
  useEffect(() => {

    fetch("http://localhost:5000/api/navigation")
      .then((res) => res.json())
      .then((data) => {

        setMenus(
          Array.isArray(data)
            ? data
            : []
        );

      })
      .catch((err) => {
        console.error(err);
      });

  }, []);

  // HANDLE INPUT CHANGES
  const handleChange = (e) => {

    const { name, value } = e.target;

    setFormData((prev) => ({
      ...prev,
      [name]: value,
    }));

  };

  const handlePageTypeChange = (e) => {

    const value = e.target.value;

    setFormData((prev) => ({
      ...prev,
      pageType: value,
    }));

  };

}
```

```

        sections: structuredClone(pageTemplates[value]),
    }));
};

// AUTO GENERATE SLUG
const handleTitleChange = (e) => {

    const title = e.target.value;

    setFormData((prev) => ({
        ...prev,

        title,

        slug: title
            .toLowerCase()
            .replace(/\s+/g, "-")
            .replace(/[^a-z0-9-]/g, "")),
    }));
};

// ADD SECTION
const addSection = (type) => {

    let newSection = {};

    switch (type) {

        case "hero":
            newSection = {
                type: "hero",
                heading: "",
                subheading: "",
            };
            break;

        case "richText":
            newSection = {
                type: "richText",
                heading: "",
                content: "",
            };
            break;

        case "list":
            newSection = {
                type: "list",
                heading: "",
                items: [""],
            };
            break;

        case "timeline":
            newSection = {
                type: "timeline",
            };
            break;
    }
};

```

```

        heading: "",
        events: [
            {
                year: "",
                text: "",
            },
        ],
    };
    break;

    default:
        return;
}

setFormData((prev) => ({
    ...prev,

    sections: [
        ...prev.sections,
        newSection,
    ],
}));
});

// UPDATE SECTION FIELD
const handleSectionChange = (
    index,
    field,
    value
) => {

    const updatedSections = [
        ...formData.sections,
    ];

    updatedSections[index][field] = value;

    setFormData((prev) => ({
        ...prev,
        sections: updatedSections,
    }));
});

// SAVE PAGE
const handleSubmit = async (e) => {

    e.preventDefault();

    // VALIDATION
    if (!formData.title.trim()) {
        alert("Page title is required");
        return;
    }
}

```

```

if (!formData.slug.trim()) {
  alert("Slug is required");
  return;
}

try {

  const response = await fetch(
    "http://localhost:5000/api/pages",
    {
      method: "POST",

      headers: {
        "Content-Type": "application/json",
      },

      body: JSON.stringify(formData),
    }
  );

  const data = await response.json();

  // HANDLE ERRORS
  if (!response.ok) {

    alert(
      data.message ||
      "Failed to create page"
    );

    return;

  }

  console.log(data);

  alert("Page created successfully!");

  // REFRESH NAVBAR
  window.dispatchEvent(
    new Event("navbarRefresh")
  );

  // RESET FORM
  // RESET FORM
  setFormData({
    title: "",
    slug: "",
    parentSlug: "",
    pageType: "generic",
    status: "published",
    sections: structuredClone(pageTemplates.generic),
  });

} catch (error) {

  console.error(error);

```

```

        alert("Failed to create page");
    }
};

return (
    <div className="max-w-3xl">

        {/* HEADER */}
        <div className="mb-8">

            <h1 className="text-3xl font-bold">
                Create Page
            </h1>

            <p className="text-gray-500 mt-1">
                Add a new website page
            </p>

        </div>

        {/* FORM */}
        <form
            onSubmit={handleSubmit}
            className="bg-white border rounded-2xl p-6 space-y-6"
        >

            {/* TITLE */}
            <div>

                <label className="block mb-2 font-medium">
                    Page Title
                </label>

                <input
                    type="text"
                    name="title"
                    value={formData.title}
                    onChange={handleTitleChange}
                    placeholder="Enter page title"
                    className="w-full border rounded-xl px-4 py-3 outline-none"
                />

            </div>

            {/* SLUG */}
            <div>

                <label className="block mb-2 font-medium">
                    Slug
                </label>

                <input
                    type="text"
                    name="slug"
                    value={formData.slug}

```

```

        onChange={handleChange}
        placeholder="page-slug"
        className="w-full border rounded-xl px-4 py-3 outline-none"
      />
    </div>

    { /* PARENT CATEGORY */ }
    <div>

      <label className="block mb-2 font-medium">
        Parent Category
      </label>

      { /* PAGE TYPE */ }
    { /* PAGE TYPE */ }
    <div>

      <label className="block mb-2 font-medium">
        Page Type
      </label>

      <select
        name="pageType"
        value={formData.pageType}
        onChange={handlePageTypeChange}
        className="w-full border rounded-xl px-4 py-3 outline-none"
      >

        <option value="generic">Generic</option>

        <option value="history">History</option>

        <option value="faculty">Faculty</option>

        <option value="non-teaching">Non Teaching</option>

        <option value="committee">Committee</option>

        <option value="gallery">Gallery</option>

        <option value="table">Table</option>

        <option value="documents">Documents</option>

        <option value="events">Events</option>

        <option value="embed">Embed</option>

        <option value="policy">Policy</option>

      </select>
    </div>

    { /* PARENT CATEGORY */ }
    <div>

      <label className="block mb-2 font-medium">
        Parent Category

```

```

</label>

<select
  name="parentSlug"
  value={formData.parentSlug}
  onChange={handleChange}
  className="w-full border rounded-xl px-4 py-3 outline-none"
>

  <option value="">
    None
  </option>

  {menus.map((menu) => (

    <option
      key={menu._id}
      value={menu.key}
    >
      {menu.title}
    </option>

  )))}

</select>
</div>

th

</div>

{/* STATUS */}
<div>

  <label className="block mb-2 font-medium">
    Status
  </label>

  <select
    name="status"
    value={formData.status}
    onChange={handleChange}
    className="w-full border rounded-xl px-4 py-3 outline-none"
  >

    <option value="published">
      Published
    </option>

    <option value="draft">
      Draft
    </option>

  </select>
</div>

```

```

{/* ADD SECTION */}
<div className="border-t pt-6">

  <h2 className="text-xl font-semibold mb-4">
    Page Sections
  </h2>

  <div className="flex flex-wrap gap-3 mb-6">

    <button
      type="button"
      onClick={() => addSection("hero")}
      className="px-4 py-2 bg-gray-100 rounded-lg"
    >
      + Hero
    </button>

    <button
      type="button"
      onClick={() => addSection("richText")}
      className="px-4 py-2 bg-gray-100 rounded-lg"
    >
      + Rich Text
    </button>

    <button
      type="button"
      onClick={() => addSection("list")}
      className="px-4 py-2 bg-gray-100 rounded-lg"
    >
      + List
    </button>

    <button
      type="button"
      onClick={() => addSection("timeline")}
      className="px-4 py-2 bg-gray-100 rounded-lg"
    >
      + Timeline
    </button>

  </div>

{/* RENDER SECTIONS */}
<div className="space-y-6">

  {formData.sections.map(
    (section, index) => (

      <div
        key={index}
        className="border rounded-2xl p-5 bg-gray-50"
      >

        <div className="mb-4">

```



```

        <h3 className="font-semibold text-lg capitalize">
            {section.type}
        </h3>

    </div>

    { /* HERO */ }
    {section.type === "hero" && (
        <div className="space-y-4">

            <input
                type="text"
                placeholder="Heading"
                value={section.heading}
                onChange={(e) =>
                    handleSectionChange(
                        index,
                        "heading",
                        e.target.value
                    )
                }
                className="w-full border rounded-xl px-4 py-3"
            />

            <textarea
                placeholder="Subheading"
                value={section.subheading}
                onChange={(e) =>
                    handleSectionChange(
                        index,
                        "subheading",
                        e.target.value
                    )
                }
                className="w-full border rounded-xl px-4 py-3 min-h-[120px]"
            />

        </div>
    )}

    { /* RICH TEXT */ }
    {section.type === "richText" && (
        <div className="space-y-4">

            <input
                type="text"
                placeholder="Heading"
                value={section.heading}
                onChange={(e) =>
                    handleSectionChange(
                        index,
                        "heading",
                        e.target.value

```

```

    )
  }
  className="w-full border rounded-xl px-4 py-3"
/>

<textarea
  placeholder="Content"
  value={section.content}
  onChange={(e) =>
    handleSectionChange(
      index,
      "content",
      e.target.value
    )
  }
  className="w-full border rounded-xl px-4 py-3 min-h-[180px]"
/>

</div>
)}

{/* LIST */}
{section.type === "list" && (
  <div className="space-y-4">

    <input
      type="text"
      placeholder="Heading"
      value={section.heading}
      onChange={(e) =>
        handleSectionChange(
          index,
          "heading",
          e.target.value
        )
      }
      className="w-full border rounded-xl px-4 py-3"
    />

    <textarea
      placeholder="Enter items separated by commas"
      value={section.items.join(", ")}
      onChange={(e) =>
        handleSectionChange(
          index,
          "items",
          e.target.value
            .split(",")
            .map((item) =>
              item.trim()
            )
        )
      }
    />
  </div>
)}

```

```

    }
    className="w-full border rounded-xl px-4 py-3 min-h-[140px]"
  />
</div>
)}

{/* TIMELINE */}
{section.type === "timeline" && (
  <div className="space-y-4">

    <input
      type="text"
      placeholder="Heading"
      value={section.heading}
      onChange={(e) =>
        handleSectionChange(
          index,
          "heading",
          e.target.value
        )
      }
      className="w-full border rounded-xl px-4 py-3"
    />

    <textarea
      placeholder="Format: 2001 - College Founded"
      value={section.events
        .map(
          (event) =>
            `${event.year} - ${event.text}`
        )
        .join("\n")}
      onChange={(e) => {
        const events =
          e.target.value
            .split("\n")
            .map((line) => {
              const parts =
                line.split("-");

              return {
                year:
                  parts[0]?.trim() || "",
                text:
                  parts
                    .slice(1)
                    .join("-")
                    .trim() || "",
              };
            )
      }
    }
  )
}

```

```

        });

        handleSectionChange(
            index,
            "events",
            events
        );
    }}
    className="w-full border rounded-xl px-4 py-3 min-h-[180px]"
  />
</div>
  )}
</div>

  )
  )}

</div>

</div>

  { /* BUTTON */}
  <button
    type="submit"
    className="bg-black text-white px-6 py-3 rounded-xl"
  >
    Save Page
  </button>

</form>

</div>
  );
};

export default CreatePage;

```